MAVEN

CHEAT SHEETS

A Complete Reference for Apache Maven Builds

Wrapper & CLI · POM · Properties · Parent POMs · Profiles · Scopes · Transitive Deps · BOMs · settings.xml · Lifecycle · Plugins · Surefire & Failsafe · Enforcer · Multi-Module · Publishing · Maven 4

24 REFERENCE SHEETS

CONTENTS

INTRODUCTION & SETUP

01 Introduction & How Maven Works
02 Installing Maven & the Wrapper
03 Project Layout & Archetypes
04 Running Maven from the CLI

THE POM

05 POM Basics & Coordinates
06 Properties & Interpolation
07 Parent POMs & Inheritance
08 Build Profiles

DEPENDENCIES

09 Declaring Dependencies & Scopes
10 Transitive Dependencies & Conflicts
11 Dependency Management & BOMs
12 Repositories & settings.xml

LIFECYCLE & PLUGINS

13 Build Lifecycles & Phases
14 Configuring Plugins
15 Compiler & Resources
16 Packaging & Running JARs

TESTING & QUALITY

17 Testing with Surefire & Failsafe
18 Code Quality & Enforcer

MULTI-MODULE & RELEASING

19 Multi-Module Projects
20 Deploying & Releasing

MAVEN 4, CI & TROUBLESHOOTING

21 What's Changing in Maven 4
22 CI & Troubleshooting

QUICK REFERENCE

23 Quick Reference: CLI & Lifecycle
24 Quick Reference: POM Snippets

MAVEN

Table of Contents

24 sheets · syntax · examples · notes

INTRODUCTION & SETUP 01 – 04

01 Introduction & How Ma…
02 Installing Maven & the …
03 Project Layout & Arch…
04 Running Maven from t…

THE POM 05 – 08

05 POM Basics & Coordin…
06 Properties & Interpolat…
07 Parent POMs & Inherit…
08 Build Profiles

DEPENDENCIES 09 – 12

09 Declaring Dependenci…
10 Transitive Dependenci…
11 Dependency Manage…
12 Repositories & setting…

LIFECYCLE & PLUGINS 13 – 16

13 Build Lifecycles & Phas…
14 Configuring Plugins
15 Compiler & Resources
16 Packaging & Running J…

TESTING & QUALITY 17 – 18

17 Testing with Surefire & …
18 Code Quality & Enforcer

MULTI-MODULE & RELEASING 19 – 20

19 Multi-Module Projects
20 Deploying & Releasing

MAVEN 4, CI & TROUBLESHOOTING 21 – 22

21 What's Changing in Ma…
22 CI & Troubleshooting

QUICK REFERENCE 23 – 24

23 Quick Reference: CLI &…
24 Quick Reference: POM …

Sheet 01

Introduction & How Maven Works

convention over configuration · POM · lifecycles & phases · plugins & goals · local vs remote repository · Maven vs Gradle

WHAT MAVEN IS

Apache **Maven** builds Java projects from a declarative `pom.xml` (Project Object Model). Instead of scripting steps, you describe *what* the project is: its coordinates, dependencies and packaging. Maven follows **convention over configuration**: a standard folder layout and a fixed **lifecycle** of phases, where each phase runs **plugin goals**. Dependencies download from **Maven Central** into a local cache at `~/.m2/repository`.

NOTES

- Current: **Maven 3.9.x / 3.10.x** (runs on Java 8+)
- **Maven 4** is at release candidate and needs Java 17+ (sheet 21)
- Everything is a plugin: `compiler`, `surefire`, `jar`, `install` …
- Central hosts most open-source JVM libraries

UNDER THE HOOD

- `mvn` starts a JVM → classworlds launcher → Maven core → resolver → plugin **mojos**
- Core plugins download on first use, which is why the very first build is slow
- Each run reads the POM, builds the **effective POM**, then plans which goals to run

A MINIMAL POM.XML

```xml
<project xmlns="http://maven.apache.org/POM/4.0.0">
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.example</groupId>      <!-- who -->
  <artifactId>hello</artifactId>      <!-- what -->
  <version>1.0.0-SNAPSHOT</version>   <!-- which -->

  <properties>
    <maven.compiler.release>21</maven.compiler.release>
  </properties>

  <dependencies>
    <dependency>
      <groupId>org.junit.jupiter</groupId>
      <artifactId>junit-jupiter</artifactId>
      <version>5.13.4</version>
      <scope>test</scope>
    </dependency>
  </dependencies>
</project>

<!-- mvn package → target/hello-1.0.0-SNAPSHOT.jar -->
```

CODE EXPLAINED

→ `groupId:artifactId:version` — the project's unique coordinates

→ `maven.compiler.release` — target Java version for `javac`

→ `<scope>test</scope>` — only on the test classpath

→ No `<build>` section needed: the defaults compile, test and jar

GRADLE EQUIVALENT

- `pom.xml` → `build.gradle.kts` + `settings.gradle.kts`
- Fixed phases → a task graph you can extend
- `~/.m2/repository` → `~/.gradle/caches`
- See Gradle 01

IDIOMATIC MAVEN

✗ ~~copying JARs into `lib/`~~ → ✓ **declare them as `<dependency>`**

✗ ~~custom folder layout~~ → ✓ **the standard `src/main/java` layout**

✗ ~~Ant-style scripted steps~~ → ✓ **plugins bound to phases**

CORE TERMS

TERM	MEANING
POM	The `pom.xml` describing the project
coordinates	groupId · artifactId · version
lifecycle	Ordered list of phases
phase	A step: compile, test, package…
plugin / goal	Code that does the work (`compiler:compile`)
repository	Where artifacts live (local or remote)

MAVEN VS GRADLE

	MAVEN	GRADLE
Build file	XML, declarative	Kotlin / Groovy code
Model	Fixed lifecycle	Task graph
Speed	Good; build-cache extension	Incremental + build cache
Learning curve	Lower	Higher
Common in	Enterprise, Spring	Android, Kotlin

⚠ COMMON MISTAKES

- Treating the POM like a script — logic belongs in plugins, not in long property hacks
- Fighting the standard layout — every plugin assumes it, so overrides multiply
- Running the system `mvn` with a different version than teammates — use the wrapper (sheet 02)

✓ TIPS & BEST PRACTICES

- Run `mvn help:effective-pom` to see what Maven will actually use
- Start from an archetype or Spring Initializr rather than a blank file
- Pin plugin versions in a parent POM so builds are repeatable

QUICK SUMMARY

- `pom.xml` — declarative project model
- `phase` — a lifecycle step; goals do the work
- `~/.m2` — local artifact cache
- `mvn package` — compile, test, jar

Sheet 02

Installing Maven & the Wrapper

SDKMAN / Homebrew · JAVA_HOME · mvn -v · mvnw / mvnw.cmd · .mvn/wrapper/maven-wrapper.properties · mvn wrapper:wrapper

INSTALL ONCE, WRAP EVERY PROJECT

Install Maven globally with a package manager so `mvn` works anywhere, then add the **Maven Wrapper** to each project. The wrapper is a pair of scripts (`mvnw`, `mvnw.cmd`) plus `.mvn/wrapper/maven-wrapper.properties` that downloads and runs the exact Maven version the project expects. Commit all three so every developer and CI server builds with the same Maven.

NOTES

- Maven needs a **JDK**, not just a JRE; it finds it via `JAVA_HOME` or `PATH`
- Wrapper 3.3+ defaults to `only-script`: no `maven-wrapper.jar` to commit
- Downloaded Maven versions land in `~/.m2/wrapper/dists`

INSTALL, VERIFY, ADD THE WRAPPER

```
sdk install maven                 # SDKMAN (macOS / Linux)
brew install maven                # Homebrew
winget install Apache.Maven       # Windows

mvn -v                            # Maven, Java version, JAVA_HOME

# add (or upgrade) the wrapper in a project
mvn wrapper:wrapper -Dmaven=3.10.0
./mvnw -v                         # first run downloads Maven
./mvnw clean verify               # use mvnw from now on
mvnw.cmd clean verify             # Windows

# .mvn/wrapper/maven-wrapper.properties
distributionUrl=https://repo.maven.apache.org/maven2/\
  org/apache/maven/apache-maven/3.10.0/apache-maven-3.10.0-bin.zip
```

CODE EXPLAINED

- → `mvn -v` — confirms which JDK Maven will compile with
- → `wrapper:wrapper` — writes `mvnw`, `mvnw.cmd` and `.mvn/wrapper/`
- → `-Dmaven=` — version to pin; re-run with a new one to upgrade
- → `distributionUrl` — the zip `mvnw` downloads; point it at a mirror if needed

GRADLE EQUIVALENT

- `mvnw` → `gradlew`
- `maven-wrapper.properties` → `gradle-wrapper.properties`
- `mvn wrapper:wrapper -Dmaven=x` → `gradle wrapper --gradle-version x`
- See Gradle 02

IDIOMATIC MAVEN

- ✗ ~~`mvn` in README and CI~~ → ✓ `./mvnw`
- ✗ ~~"install Maven 3.x first" docs~~ → ✓ **commit the wrapper and say nothing**
- ✗ ~~a committed `maven-wrapper.jar`~~ → ✓ `only-script` **wrapper type**

WRAPPER TYPES (-DTYPE=)

TYPE	WHAT IT COMMITS
`only-script`	Scripts only; downloads Maven (default)
`script`	Scripts + downloads a wrapper JAR
`bin`	Scripts + `maven-wrapper.jar`
`source`	Scripts + Java source of the downloader

WRAPPER FILES

FILE	COMMIT?
`mvnw`	✔ (keep it executable)
`mvnw.cmd`	✔
`.mvn/wrapper/maven-wrapper.properties`	✔
`~/.m2/wrapper/dists/`	✖ local cache

⚠ COMMON MISTAKES

- `JAVA_HOME` pointing at an old JDK — `mvn -v` shows it; builds fail with "release 21 not supported"
- Committing `mvnw` without the execute bit — run `git update-index --chmod=+x mvnw`
- Upgrading Maven on your machine and assuming CI follows — only the wrapper version counts

✓ TIPS & BEST PRACTICES

- Add `distributionSha256Sum` to the properties file to verify the download
- Corporate proxy? Set `MVNW_REPOURL` to your internal mirror
- Keep a global Maven only for creating new projects; build with `./mvnw`

QUICK SUMMARY

- `mvn -v` — check Maven and JDK
- `mvn wrapper:wrapper` — add the wrapper
- `./mvnw` — the command to use everywhere
- `distributionUrl` — pins the Maven version

Sheet 03

Project Layout & Archetypes

archetype:generate · maven-archetype-quickstart · src/main/java · src/test/java · src/main/resources · target/ · .mvn/

THE STANDARD DIRECTORY LAYOUT

Every Maven plugin assumes the same folders, so a project that follows them needs almost no configuration. Production code goes in `src/main/java`, tests in `src/test/java`, and non-code files in the matching `resources` folders. Build output goes to `target/`. An **archetype** is a project template: `archetype:generate` creates this layout with a starter POM.

NOTES

- `target/` is disposable; add it to `.gitignore`
- Package folders mirror the Java package: `com/example/app`
- `.mvn/` holds the wrapper, `maven.config` and `extensions.xml`
- For Spring Boot, start.spring.io generates the same layout

GENERATE A PROJECT

```
mvn archetype:generate \
  -DgroupId=com.example \
  -DartifactId=my-app \
  -DarchetypeArtifactId=maven-archetype-quickstart \
  -DarchetypeVersion=1.5 \
  -DinteractiveMode=false

cd my-app && tree
my-app/
├── pom.xml
└── src/
    ├── main/java/com/example/App.java
    └── test/java/com/example/AppTest.java

mvn archetype:generate          # interactive: pick from a list
mvn package                     # creates target/
ls target/
#  classes/  test-classes/  surefire-reports/
#  my-app-1.0-SNAPSHOT.jar
```

CODE EXPLAINED

- → `-DarchetypeArtifactId` — which template to use
- → `-DarchetypeVersion` — pin it; old defaults target ancient Java
- → `-DinteractiveMode=false` — no prompts (scripts, CI)
- → `target/classes` — compiled main code and copied resources

GRADLE EQUIVALENT

- `archetype:generate` → `gradle init`
- Same `src/main/java` / `src/test/java` layout
- `target/` → `build/`
- See Gradle 03

IDIOMATIC MAVEN

- ✗ ~~`<sourceDirectory>src</sourceDirectory>`~~ → ✓ **move files into** `src/main/java`
- ✗ ~~config files next to classes~~ → ✓ `src/main/resources`
- ✗ ~~test data in `src/main`~~ → ✓ `src/test/resources`

STANDARD FOLDERS

FOLDER	HOLDS
`src/main/java`	Application source
`src/main/resources`	Config, templates, SQL (on classpath)
`src/test/java`	Test source
`src/test/resources`	Test-only files
`src/main/webapp`	Web content (war packaging)
`target/`	All build output

USEFUL ARCHETYPES

ARCHETYPEARTIFACTID	CREATES
`maven-archetype-quickstart`	Plain Java app + JUnit test
`maven-archetype-webapp`	war project with web.xml
`maven-archetype-archetype`	Your own archetype
`maven-archetype-simple`	Minimal jar project

⚠ COMMON MISTAKES

- Running `archetype:generate` without `-DarchetypeVersion` — you may get an old template with Java 1.7 settings
- Putting `application.properties` in `src/main/java` — it never reaches the classpath
- Committing `target/` — it is regenerated on every build

✓ TIPS & BEST PRACTICES

- After generating, update the JUnit version and set `maven.compiler.release`
- Create your own archetype from an existing project: `mvn archetype:create-from-project`
- Use `-B` (batch mode) in scripts to avoid any prompt

QUICK SUMMARY

- `archetype:generate` — scaffold a project
- `src/main` / `src/test` — code and resources
- `target/` — build output, never committed
- `.mvn/` — per-project Maven settings

Sheet 04

Running Maven from the CLI

phases vs plugin:goal · mvn clean install · -D properties · -P profiles · -o offline · -q / -X · -B batch mode · -T threads

PHASES AND GOALS

You can give `mvn` two kinds of arguments. A **phase** (`package`) runs that phase and every phase before it. A **goal** (`dependency:tree`) runs one plugin goal on its own. You can mix them, and they run left to right. Options start with –: –D sets a property, –P picks a profile, `-pl` picks modules.

NOTES

- `mvn clean verify` — the safest full build
- Goals use `prefix:goal` or full `groupId:artifactId:version:goal`
- Put default options in `.mvn/maven.config`

UNDER THE HOOD

- A phase runs **every earlier phase**; `plugin:goal` runs only that goal
- `-T` builds independent reactor modules in parallel, not steps inside one module
- `.mvn/maven.config` and `.mvn/jvm.config` are read at startup

EVERYDAY COMMANDS

```
mvn compile                    # validate → compile
mvn test                       # … → test
mvn package                    # … → jar in target/
mvn verify                     # … → integration tests, checks
mvn install                    # … → copy to ~/.m2
mvn clean verify               # wipe target/ first

mvn dependency:tree            # one goal, no lifecycle
mvn clean package dependency:tree    # mix, left to right

mvn verify -DskipTests         # -D sets a property
mvn verify -Pprod              # activate a profile
mvn -o verify                  # offline (cache only)
mvn -U verify                  # re-check SNAPSHOTs / missing
mvn -q verify                  # quiet: errors only
mvn -X verify                  # debug logging
mvn -B -ntp verify             # CI: batch, no download noise
mvn -T 1C verify               # 1 thread per CPU core
mvn -f other/pom.xml verify    # use another POM
```

CODE EXPLAINED

→ `mvn test` — also runs `compile`, `process-resources` …

→ `dependency:tree` — prefix `dependency` → `maven-dependency-plugin`

→ `-U` — forces update checks for SNAPSHOTs and failed lookups

→ `-ntp` — `--no-transfer-progress`

GRADLE EQUIVALENT

- `mvn package` → `./gradlew assemble`; `verify` → `build`
- `-DskipTests` → `-x test`
- `-Pprofile` → no equivalent; use `-Pname=value` properties
- See Gradle 04 · Gradle 26

IDIOMATIC MAVEN

✗ ~~`mvn clean install` by habit~~ → ✓ `mvn verify` **unless another build needs the JAR**

✗ ~~long flag lists in every command~~ → ✓ `.mvn/maven.config`

✗ ~~`-Dmaven.test.skip=true` to save time~~ → ✓ `-DskipTests` **(tests still compile)**

COMMON FLAGS

FLAG	EFFECT
`-D name=value`	Set a user / system property
`-P a,!b`	Activate a, deactivate b
`-pl / -am / -amd`	Select modules (sheet 19)
`-rf :module`	Resume from a module
`-fae / -ff`	Fail at end / fail fast
`-e`	Show full stack traces
`-N`	Non-recursive: root POM only

.MVN/ CONFIG FILES

FILE	HOLDS
`maven.config`	CLI options, one per line: –T1C
`jvm.config`	JVM flags: –Xmx2g
`extensions.xml`	Core extensions (build cache…)
`wrapper/`	Wrapper properties

⚠ COMMON MISTAKES

- Thinking `mvn install` is needed to run tests — `mvn test` or `verify` is enough
- Running `mvn jar:jar` and expecting a compile first — a lone goal skips the lifecycle
- Using `-o` on a fresh machine — offline mode fails on anything not already cached

✓ TIPS & BEST PRACTICES

- Shell autocompletion: the `maven` plugin for oh-my-zsh, or bash-completion scripts
- Find a goal's parameters: `mvn help:describe -Dplugin=surefire -Ddetail`
- Add `-Dstyle.color=always` when CI strips colors you want

QUICK SUMMARY

- `phase` — runs itself and all earlier phases
- `plugin:goal` — runs one goal only
- `-D / -P / -pl` — properties, profiles, modules
- `-B -ntp` — clean CI output

POM Basics & Coordinates

COORDINATES IDENTIFY AN ARTIFACT

Every artifact in the Maven world is identified by its **coordinates**: `groupId` (organization, reverse domain), `artifactId` (project name) and `version`. `packaging` decides what gets built and which plugins run (`jar` by default). A version ending in `-SNAPSHOT` means "in development" and can change. A version without it is a **release** and must never change once published.

NOTES

- Written as `groupId:artifactId:version` (GAV)
- File name in a repo: `artifactId-version.jar`
- `modelVersion` is always `4.0.0` in Maven 3
- `name`, `url`, `licenses` are needed to publish to Central

THE TOP OF A POM

```xml
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
           https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.acme.billing</groupId>
  <artifactId>invoice-service</artifactId>
  <version>2.4.0-SNAPSHOT</version>
  <packaging>jar</packaging>          <!-- default -->

  <name>Invoice Service</name>
  <description>Creates and sends invoices</description>
  <url>https://github.com/acme/invoice-service</url>

  <!-- dependencies, build, profiles … -->
</project>
<!-- → com/acme/billing/invoice-service/2.4.0-SNAPSHOT/
       invoice-service-2.4.0-SNAPSHOT.jar -->
```

CODE EXPLAINED

- → `xsi:schemaLocation` — gives the IDE completion and validation
- → `groupId` dots become folders in the repository
- → `-SNAPSHOT` — rebuilt and re-downloaded as it changes
- → `packaging` — `jar`, `war`, `pom`, `maven-plugin` …

GRADLE EQUIVALENT

- `groupId` → `group = "com.acme.billing"`
- `artifactId` → `rootProject.name` / project name
- `version` → `version = "2.4.0-SNAPSHOT"`
- See Gradle 03 · Gradle 09

IDIOMATIC MAVEN

- ✗ ~~`groupId` = `billing`~~ → ✓ **reverse domain:** `com.acme.billing`
- ✗ ~~re-publishing `1.2.0` with a fix~~ → ✓ **release** `1.2.1`
- ✗ ~~`1.0`, `1.0-final`, `v1`~~ → ✓ **semantic** `MAJOR.MINOR.PATCH`

PACKAGING TYPES

PACKAGING	PRODUCES
`jar`	Library or app JAR (default)
`war`	Web archive for a servlet container
`pom`	Parent / aggregator, no code
`maven-plugin`	A Maven plugin
`ear`	Enterprise archive

VERSION ORDER

VERSION	NOTES
`1.0-alpha-1`	Qualifiers sort before the release
`1.0-beta-2`	
`1.0-RC1`	Release candidate
`1.0-SNAPSHOT`	Before 1.0, mutable
`1.0`	Release, immutable
`1.0.1`	Patch

⚠ COMMON MISTAKES

- Depending on another team's `-SNAPSHOT` in a release — the build changes under you
- Mismatched `artifactId` and folder name in multi-module builds — confusing errors
- Using uppercase or spaces in `artifactId` — keep it lowercase-hyphenated

✓ TIPS & BEST PRACTICES

- Check what a POM resolves to: `mvn help:evaluate -Dexpression=project.version -q -DforceStdout`
- Keep `-SNAPSHOT` on `main`; strip it only when releasing
- Add the schema header so your IDE flags typos in element names

QUICK SUMMARY

- `groupId:artifactId:version` — unique coordinates
- `packaging` — what gets built
- `-SNAPSHOT` — mutable dev version
- Release — immutable once published

Properties & Interpolation

REUSABLE VALUES WITH ${…}

Define values once in `<properties>` and reference them anywhere with `${name}`. Maven also exposes built-in values: `${project.*}` for POM fields, `${env.*}` for environment variables, `${settings.*}` for `settings.xml`, and Java system properties like `${user.home}`. Many plugins read **user properties** directly (`maven.compiler.release`, `skipTests`), so setting a property is often all the configuration needed.

NOTES

- `-Dname=value` on the command line overrides the POM
- Properties are inherited from parent POMs
- Use properties for versions shared by several dependencies

DEFINING AND USING PROPERTIES

```
<properties>
  <!-- read by plugins -->
  <maven.compiler.release>21</maven.compiler.release>
  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>

  <!-- your own -->
  <jackson.version>2.19.2</jackson.version>
  <app.env>dev</app.env>
</properties>

<dependency>
  <groupId>com.fasterxml.jackson.core</groupId>
  <artifactId>jackson-databind</artifactId>
  <version>${jackson.version}</version>
</dependency>

<build>
  <finalName>${project.artifactId}-${app.env}</finalName>
</build>

<!-- mvn package -Dapp.env=prod → invoice-service-prod.jar -->
<!-- mvn help:evaluate -Dexpression=jackson.version -q -DforceStdout -->
```

CODE EXPLAINED

- → `maven.compiler.release` — the compiler plugin reads it, no plugin block needed
- → `sourceEncoding` — stops "platform encoding" warnings
- → `${jackson.version}` — one place to upgrade all Jackson modules
- → `-Dapp.env=prod` — overrides the POM value for this run

GRADLE EQUIVALENT

- `<properties>` → `gradle.properties` or `extra["x"]`
- `${jackson.version}` → version catalog `version.ref`
- `-Dname=value` → `-Pname=value` + `providers.gradleProperty`
- See Gradle 08

IDIOMATIC MAVEN

- ✗ ~~`${pom.version}`~~ → ✓ `${project.version}`
- ✗ ~~source~~ + ~~target~~ ~~properties~~ → ✓ `maven.compiler.release`
- ✗ ~~a version repeated in 5 dependencies~~ → ✓ **a BOM or one property (sheet 11)**

BUILT-IN EXPRESSIONS

EXPRESSION	VALUE
`${project.version}`	This POM's version
`${project.basedir}`	Folder with pom.xml
`${project.build.directory}`	`target`
`${project.build.finalName}`	artifactId-version
`${env.HOME}`	Environment variable
`${user.home}`	Java system property
`${settings.localRepository}`	`~/.m2/repository`

PLUGIN USER PROPERTIES

PROPERTY	PLUGIN EFFECT
`maven.compiler.release`	javac `--release`
`skipTests`	Surefire skips running tests
`maven.test.skip`	Skip compiling and running tests
`maven.javadoc.skip`	Skip javadoc
`gpg.skip`	Skip signing
`revision / sha1 / changelist`	CI-friendly versions (sheet 19)

⚠ COMMON MISTAKES

- Expecting `${name}` to work in `src/main/resources` — only with resource filtering (sheet 15)
- Using properties in `groupId` / `artifactId` — Maven warns; only `${revision}`-style versions are supported
- Relying on `${env.X}` that CI doesn't set — the literal `${env.X}` leaks into output

✓ TIPS & BEST PRACTICES

- Name your own properties with a dot prefix (`app.`) to avoid clashing with plugin properties
- See every resolved property: `mvn help:effective-pom -Dverbose`
- Prefer `maven.compiler.release` over plugin config in simple projects

QUICK SUMMARY

- `<properties>` — define once
- `${name}` — reference anywhere in the POM
- `-Dname=value` — override per run
- `${project.*}` / `${env.*}` — built-ins

Sheet 07

Parent POMs & Inheritance

<parent> · relativePath · Super POM · what gets inherited · help:effective-pom · spring-boot-starter-parent

SHARING CONFIGURATION DOWN

A POM can name a **parent** and inherit its properties, dependency management, plugin configuration and repositories. Every POM ultimately inherits from the built-in **Super POM**, which supplies the standard layout and Central. A parent must have `packaging pom`. Use a company parent for shared rules, or a framework parent like `spring-boot-starter-parent`.

NOTES

- `relativePath` defaults to `../pom.xml`
- Use `<relativePath/>` for a parent from a repository
- Only one parent; share more through BOM imports (sheet 11)

UNDER THE HOOD

- Effective POM = Super POM + parent chain + active profiles, merged, then `${}` interpolated
- Plugins merge by `groupId:artifactId`; config elements merge, they aren't appended
- `combine.children="append"` / `combine.self="override"` change the merge

A CHILD POM

```
<parent>
  <groupId>com.acme</groupId>
  <artifactId>acme-parent</artifactId>
  <version>7</version>
  <relativePath/>        <!-- look it up in repositories -->
</parent>

<artifactId>invoice-service</artifactId>
<!-- groupId and version inherited from the parent -->

<build>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-compiler-plugin</artifactId>
      <configuration>
        <compilerArgs combine.children="append">
          <arg>-parameters</arg>    <!-- added to the parent's args -->
        </compilerArgs>
      </configuration>
    </plugin>
  </plugins>
</build>

<!-- mvn help:effective-pom -Doutput=effective.xml -->
```

CODE EXPLAINED

→ `<relativePath/>` — skip the `../pom.xml` lookup

→ No `<version>` on the plugin — it comes from the parent's `pluginManagement`

→ `combine.children="append"` — keeps the parent's list items too

→ `help:effective-pom` — the merged result Maven actually runs

GRADLE EQUIVALENT

- Parent POM → a convention plugin in `build-logic`
- Super POM defaults → the `java` plugin's conventions
- Several parents → apply several convention plugins
- See Gradle 18

IDIOMATIC MAVEN

✗ ~~dependencies in a parent POM~~ → ✓ **`dependencyManagement` in the parent**

✗ ~~copy-pasted plugin blocks~~ → ✓ **one `pluginManagement` in a parent**

✗ ~~a parent only for versions~~ → ✓ **a BOM import**

INHERITED?

ELEMENT	INHERITED
`groupId, version`	✔
`properties`	✔
`dependencies`	✔ (every child gets them)
`dependencyManagement`	✔
`build plugins / pluginManagement`	✔
`artifactId, name`	✖
`profiles`	✖ (effects apply when active)

WELL-KNOWN PARENTS

PARENT	PROVIDES
`spring-boot-starter-parent`	Boot versions, plugin config
`org.apache:apache`	Apache project defaults
`company parent`	Your rules, repos, plugins

⚠ COMMON MISTAKES

- Putting `<dependencies>` in a parent — every child gets them, wanted or not
- Forgetting `<relativePath/>` — Maven warns that `../pom.xml` isn't the parent
- Changing parent config and not seeing it — the child has a local copy of the old parent; run `install` on the parent

✓ TIPS & BEST PRACTICES

- Debug any "where did this come from" question with `help:effective-pom -Dverbose`
- Version company parents with plain integers (`7`, `8`) and release them often
- Use `<inherited>false</inherited>` for plugin config only the parent itself needs

QUICK SUMMARY

- `<parent>` — inherit config
- `relativePath` — where to find it locally
- Super POM — the root of every POM
- `help:effective-pom` — the merged result

Build Profiles

CONDITIONAL CONFIGURATION

A **profile** is a named block of POM settings that applies only when activated: on the command line with –P, or automatically through an `<activation>` condition (JDK version, OS, a property, or a file). Profiles can change properties, dependencies, plugins and modules. Use them for real environmental differences (OS, JDK, CI), not as a way to build different apps from one POM.

NOTES

- Profiles can live in the POM or in `settings.xml`
- `activeByDefault` turns off as soon as another profile in that POM is activated
- Several conditions in one `<activation>` must all match (Maven 3.2.2+)

PROFILES WITH ACTIVATION

```
<profiles>
  <profile>
    <id>ci</id>
    <activation>
      <property><name>env.CI</name></property>    <!-- CI sets CI=true -->
    </activation>
    <properties><skipITs>false</skipITs></properties>
  </profile>

  <profile>
    <id>windows</id>
    <activation><os><family>windows</family></os></activation>
    <properties><script.ext>.cmd</script.ext></properties>
  </profile>

  <profile>
    <id>jdk25</id>
    <activation><jdk>[25,)</jdk></activation>
  </profile>

  <profile>
    <id>native</id>                  <!-- only with -Pnative -->
    <build><plugins>…</plugins></build>
  </profile>
</profiles>
<!-- mvn verify -Pnative,!ci   ·   mvn help:active-profiles -->
```

CODE EXPLAINED

- → `<property><name>env.CI` — active when the env var exists
- → `<os><family>` — `windows`, `mac`, `unix`
- → `[25,)` — version range: JDK 25 or newer
- → `-P native,!ci` — turn one on, one off

GRADLE EQUIVALENT

- No profiles: use `if (providers.gradleProperty("ci").isPresent)`
- `-Pnative` → `-Pnative=true` read as a property
- OS / JDK checks → plain Kotlin conditions in the build script
- See Gradle 08

IDIOMATIC MAVEN

- ✗ ~~`dev` / `prod` profiles building different JARs~~ → ✓ **one artifact + runtime config**
- ✗ ~~`activeByDefault`~~ → ✓ **put the default config outside any profile**
- ✗ ~~secrets in a POM profile~~ → ✓ **a `settings.xml` profile or env vars**

ACTIVATION CONDITIONS

ELEMENT	ACTIVE WHEN
`<jdk>21</jdk>`	JDK version starts with 21
`<jdk>[17,21)</jdk>`	Range: 17 up to, not incl., 21
`<os>…</os>`	family, name, arch, version
`<property> name`	Property is set
`<property> name + value`	Property equals value (`!value` negates)
`<file><exists>`	File exists / `<missing>`

COMMANDS

COMMAND	SHOWS / DOES
`-P a,b`	Activate a and b
`-P !a (or -P -a)`	Deactivate a
`help:active-profiles`	Which profiles are on
`help:all-profiles`	Every profile available

⚠ COMMON MISTAKES

- Relying on `activeByDefault` — it silently switches off when you add `-Pother`
- Typing `-P!ci` in bash — `!` triggers history expansion; quote it: `-P '!ci'`
- Profiles that change dependencies of a published library — consumers never see them

✓ TIPS & BEST PRACTICES

- Keep profiles few and named for the condition (`ci`, `windows`), not the person
- Check activation before debugging anything else: `mvn help:active-profiles`
- Use a `settings.xml` profile for per-machine repositories or credentials

QUICK SUMMARY

- `<profile>` — conditional POM settings
- `<activation>` — jdk, os, property, file
- `-P id / !id` — on / off manually
- `help:active-profiles` — check what's on

Declaring Dependencies & Scopes

DEPENDENCIES AND THEIR SCOPE

Each <dependency> names an artifact by its coordinates and a **scope** that decides which classpaths it lands on and whether it passes on to projects that depend on yours. `compile` is the default and is visible everywhere. `provided` is for things the runtime supplies (the Servlet API, Lombok). `runtime` is needed only to run (JDBC drivers), and `test` only for tests.

NOTES

- Omit `<version>` when a BOM or parent manages it
- `<optional>true</optional>` — used by you, not passed on
- `classifier` picks a variant: `sources`, `tests`, `linux-x86_64`
- Check the result: `mvn dependency:tree`

A DEPENDENCIES BLOCK

```
<dependencies>
  <dependency>                              <!-- compile (default) -->
    <groupId>com.google.guava</groupId>
    <artifactId>guava</artifactId>
    <version>33.4.8-jre</version>
  </dependency>
  <dependency>                              <!-- container supplies it -->
    <groupId>jakarta.servlet</groupId>
    <artifactId>jakarta.servlet-api</artifactId>
    <version>6.1.0</version>
    <scope>provided</scope>
  </dependency>
  <dependency>                              <!-- needed only to run -->
    <groupId>org.postgresql</groupId>
    <artifactId>postgresql</artifactId>
    <version>42.7.7</version>
    <scope>runtime</scope>
  </dependency>
  <dependency>                              <!-- version from junit-bom -->
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter</artifactId>
    <scope>test</scope>
  </dependency>
</dependencies>
```

CODE EXPLAINED

- → `compile` — main + test classpath, and passed to consumers
- → `provided` — compile only; not packaged, not passed on
- → `runtime` — not visible to `javac`, only when running
- → No version on `junit-jupiter` — a BOM supplies it (sheet 11)

GRADLE EQUIVALENT

- `compile` → `implementation` (or `api`)
- `provided` → `compileOnly`; `runtime` → `runtimeOnly`
- `test` → `testImplementation`
- See Gradle 09

IDIOMATIC MAVEN

- ✗ ~~`<scope>system</scope>`~~ + ~~`systemPath`~~ → ✓ **install the JAR to a repository**
- ✗ ~~JDBC driver at~~ ~~`compile`~~ → ✓ `runtime` **scope**
- ✗ ~~Lombok at~~ ~~`compile`~~ → ✓ `provided` **+ annotation processor path**

SCOPES

SCOPE	COMPILE	TEST	RUNTIME	TRANSITIVE
compile	✔	✔	✔	✔
provided	✔	✔	✖	✖
runtime	✖	✔	✔	✔
test	✖	✔	✖	✖
system	✔	✔	✖	✖
import	BOMs only (sheet 11)			

OTHER DEPENDENCY ELEMENTS

ELEMENT	USE
<classifier>	Variant: `sources`, `tests`, `jdk8`
<type>	Packaging: `jar` (default), `pom`, `test-jar`
<optional>	Don't pass to consumers
<exclusions>	Drop a transitive (sheet 10)

⚠ COMMON MISTAKES

- Using `provided` for things that must ship in a Spring Boot fat JAR — they're missing at runtime
- Leaving JUnit at `compile` scope — it ends up in your published dependencies
- Copy-pasting a dependency with the wrong `groupId` (old `javax.*` vs `jakarta.*`)

✓ TIPS & BEST PRACTICES

- Search coordinates on central.sonatype.com and copy the Maven snippet
- Find unused or undeclared dependencies: `mvn dependency:analyze`
- Mark plugin-style extras `<optional>` so consumers don't inherit them

QUICK SUMMARY

- `compile` — default, everywhere, transitive
- `provided` — compile only, supplied at runtime
- `runtime` — run only
- `test` — tests only

Transitive Dependencies & Conflicts

nearest-wins mediation · <exclusions> · dependency:tree · -Dverbose · dependency:analyze · enforcer dependencyConvergence

DEPENDENCIES OF DEPENDENCIES

When you depend on a library, you also get **its** dependencies, and theirs. When two paths bring different versions of the same artifact, Maven picks one by **dependency mediation**: the version **nearest** to your project in the tree wins, and the first one declared wins a tie. It does *not* pick the highest version, so an old version can silently win. `dependency:tree` shows what happened.

NOTES

- Declaring a dependency directly always wins (depth 1)
- `dependencyManagement` overrides mediation (sheet 11)
- `-Dverbose` shows the versions that lost

UNDER THE HOOD

- Mediation: nearest, then first declared, never highest
- `optional`, `test` and `provided` dependencies don't propagate
- A transitive dependency's scope comes from its parent's: `test` → everything below is `test`

READING THE TREE, EXCLUDING

```
$ mvn dependency:tree -Dverbose -Dincludes=com.fasterxml.jackson.core
[INFO] com.acme:invoice-service:jar:2.4.0-SNAPSHOT
[INFO] +- com.acme:pdf-lib:jar:1.3.0:compile
[INFO] |  \- com.fasterxml.jackson.core:jackson-databind:jar:2.12.7:compile
[INFO] \- com.acme:http-lib:jar:4.0.0:compile
[INFO]    \- (com.fasterxml.jackson.core:jackson-databind:jar:2.19.2:compile
              - omitted for conflict with 2.12.7)
```

```
<!-- fix 1: drop the old one -->
<dependency>
  <groupId>com.acme</groupId>
  <artifactId>pdf-lib</artifactId>
  <version>1.3.0</version>
  <exclusions>
    <exclusion>
      <groupId>com.fasterxml.jackson.core</groupId>
      <artifactId>jackson-databind</artifactId>
    </exclusion>
  </exclusions>
</dependency>
<!-- fix 2 (better): pin it in dependencyManagement -->
```

CODE EXPLAINED

- → `-Dincludes=` — filter the tree to one group or artifact
- → `omitted for conflict` — 2.19.2 lost; 2.12.7 was declared first at the same depth
- → `<exclusion>` — removes that artifact from this path only
- → `*` works as a wildcard for `groupId` / `artifactId`

GRADLE EQUIVALENT

- Nearest wins → Gradle picks the **highest** version by default
- `<exclusions>` → `exclude(group = "…", module = "…")`
- `dependency:tree` → `./gradlew dependencies` / `dependencyInsight`
- See Gradle 12

IDIOMATIC MAVEN

- ✗ ~~excluding from every dependency that pulls it~~ → ✓ **one entry in** `dependencyManagement`
- ✗ ~~ignoring conflicts until runtime~~ → ✓ `enforcer` `dependencyConvergence`
- ✗ ~~using a class you get transitively~~ → ✓ **declare it directly**

RESULTING TRANSITIVE SCOPE

DIRECT ↓ / TRANSITIVE →	COMPILE	RUNTIME
compile	compile	runtime
provided	provided	provided
runtime	runtime	runtime
test	test	test

DIAGNOSE

COMMAND	SHOWS
`dependency:tree`	Resolved tree
`dependency:tree -Dverbose`	Plus omitted / duplicate versions
`dependency:analyze`	Used-undeclared, unused-declared
`dependency:list`	Flat resolved list
`enforcer:enforce`	Fails on rule violations

⚠ COMMON MISTAKES

- Assuming the newest version wins — Maven's nearest-wins can pick an old one and cause `NoSuchMethodError`
- Excluding a library that the dependency really needs — runtime `ClassNotFoundException`
- Fixing conflicts by reordering dependencies — fragile; pin the version instead

✓ TIPS & BEST PRACTICES

- Add the enforcer's `dependencyConvergence` or `requireUpperBoundDeps` rule to catch conflicts early (sheet 18)
- IntelliJ's Maven dependency diagram shows conflicts in red
- When `NoSuchMethodError` appears, run `dependency:tree -Dverbose -Dincludes=…` first

QUICK SUMMARY

- Transitive — you get your dependencies' dependencies
- Mediation — nearest, then first declared
- `<exclusions>` — drop one path
- `dependency:tree -Dverbose` — see conflicts

Sheet 11

Dependency Management & BOMs

<dependencyManagement> · version alignment · scope import · type pom · junit-bom · spring-boot-dependencies · versions:display-dependency-updates

PIN VERSIONS IN ONE PLACE

<dependencyManagement> declares versions (and optionally scopes and exclusions) **without adding** the dependency. Any <dependency> for that artifact, direct or transitive, then uses the managed version, so you can leave <version> out. A **BOM** (bill of materials) is a POM full of managed versions. Import it with `scope import` to align a whole family of libraries (JUnit, Jackson, Spring) at once.

NOTES

- Managed versions also apply to **transitive** dependencies
- Import BOMs inside `dependencyManagement` only
- A version written on a `<dependency>` beats the managed one

UNDER THE HOOD

- BOM imports are merged when the model is built; the first import wins a tie
- Your own `dependencyManagement` entries beat any imported BOM
- Imports aren't inherited as imports; their managed entries are

IMPORTING BOMS

```
<dependencyManagement>
  <dependencies>
    <dependency>                          <!-- pin one artifact -->
      <groupId>com.fasterxml.jackson.core</groupId>
      <artifactId>jackson-databind</artifactId>
      <version>2.19.2</version>
    </dependency>
    <dependency>                          <!-- import a BOM -->
      <groupId>org.junit</groupId>
      <artifactId>junit-bom</artifactId>
      <version>5.13.4</version>
      <type>pom</type>
      <scope>import</scope>
    </dependency>
  </dependencies>
</dependencyManagement>

<dependencies>
  <dependency>                            <!-- no versions needed -->
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter</artifactId>
    <scope>test</scope>
  </dependency>
</dependencies>
```

CODE EXPLAINED

- → `jackson-databind` — every path that pulls it now gets 2.19.2
- → `<type>pom` + `<scope>import` — both required for a BOM
- → `junit-jupiter` — version filled in from `junit-bom`
- → Order matters: put the BOM you trust most first

GRADLE EQUIVALENT

- `dependencyManagement` → `constraints { }`
- BOM import → `implementation(platform("org.junit:junit-bom:5.13.4"))`
- Version properties → version catalog `[versions]`
- See Gradle 11 · Gradle 12

IDIOMATIC MAVEN

- ✗ ~~`<version>` on every child dependency~~ → ✓ **manage it once in the parent**
- ✗ ~~separate versions per Jackson module~~ → ✓ **import** `jackson-bom`
- ✗ ~~hand-checking for updates~~ → ✓ `versions:display-dependency-updates`

COMMON BOMS

BOM	ALIGNS
`org.junit:junit-bom`	JUnit Jupiter / Platform
`com.fasterxml.jackson:jackson-bom`	Jackson modules
`org.springframework.boot:spring-boot-dependencies`	Everything Boot manages
`io.netty:netty-bom`	Netty
`org.testcontainers:testcontainers-bom`	Testcontainers

VERSIONS-MAVEN-PLUGIN

GOAL	DOES
`display-dependency-updates`	List newer dependency versions
`display-plugin-updates`	List newer plugin versions
`display-property-updates`	Check `${x.version}` properties
`update-properties`	Bump those properties

⚠ COMMON MISTAKES

- Putting a BOM under `<dependencies>` instead of `dependencyManagement` — it does nothing useful
- Forgetting `<type>pom</type>` — Maven looks for a JAR and fails
- Two BOMs managing the same artifact — the first import wins, which may not be the one you meant

✓ TIPS & BEST PRACTICES

- Override one version from an imported BOM by declaring it in your own `dependencyManagement`
- Publish your own BOM (`packaging pom`) for multi-module libraries
- Spring Boot without the parent: import `spring-boot-dependencies` as a BOM

QUICK SUMMARY

- `dependencyManagement` — versions without adding deps
- `scope import` — pull in a BOM
- Managed versions — apply to transitives too
- BOM — aligns a library family

Repositories & settings.xml

WHERE ARTIFACTS COME FROM

Maven resolves artifacts from the **local repository** (`~/.m2/repository`) first, then from **remote repositories**: Maven Central by default, plus any declared in the POM. Machine-specific settings live in `~/.m2/settings.xml`: **mirrors** that redirect traffic to a company proxy (Nexus, Artifactory), **server** credentials, and proxies. Keep credentials out of the POM.

NOTES

- Global settings: `$MAVEN_HOME/conf/settings.xml`
- A `<server>` `id` must match the repository or mirror `id`
- Plain `http://` repositories are blocked since Maven 3.8.1

UNDER THE HOOD

- Lookup order: local repo → mirror (if it matches) → repositories in the POM
- `_remote.repositories` records where each file came from; `*.lastUpdated` marks failed lookups
- SNAPSHOT update policy defaults to **daily**; `-U` forces a check

~/.M2/SETTINGS.XML

```
<settings>
  <mirrors>
    <mirror>
      <id>corp</id>
      <mirrorOf>*</mirrorOf>          <!-- route ALL repos here -->
      <url>https://nexus.acme.com/repository/maven-public/</url>
    </mirror>
  </mirrors>

  <servers>
    <server>
      <id>corp</id>                   <!-- matches the mirror id -->
      <username>${env.NEXUS_USER}</username>
      <password>${env.NEXUS_TOKEN}</password>
    </server>
  </servers>
</settings>

<!-- pom.xml: an extra repository -->
<repositories>
  <repository>
    <id>confluent</id>
    <url>https://packages.confluent.io/maven/</url>
    <snapshots><enabled>false</enabled></snapshots>
  </repository>
</repositories>
```

CODE EXPLAINED

→ `<mirrorOf>*` — every repository request goes to Nexus

→ `${env.NEXUS_TOKEN}` — credentials from the environment, not the file

→ `<snapshots><enabled>false` — releases only from this repo

→ Use `mvn -s ci-settings.xml` for a CI-specific file

GRADLE EQUIVALENT

- Central → `mavenCentral()`; extra repo → `maven { url = uri("…") }`
- Mirrors → `dependencyResolutionManagement` in settings
- `~/.m2/repository` → `mavenLocal()` (avoid in Gradle)
- See Gradle 10

IDIOMATIC MAVEN

✗ ~~passwords in `pom.xml`~~ → ✓ `<servers>` **in** `settings.xml`

✗ ~~plain-text passwords in settings~~ → ✓ **env vars or** `mvn --encrypt-password`

✗ ~~`<repositories>` in every project~~ → ✓ **one company mirror**

MIRROROF PATTERNS

PATTERN	MATCHES
`*`	Every repository
`external:*`	All except localhost / file repos
`central`	Only Maven Central
`*,!confluent`	All except `confluent`

LOCAL REPO COMMANDS

COMMAND	DOES
`dependency:go-offline`	Download everything for –o
`dependency:purge-local-repository`	Delete and re-resolve this project's deps
`-Dmaven.repo.local=path`	Use another local repo
`--encrypt-master-password`	Create `settings-security.xml`

⚠ COMMON MISTAKES

- A `<server>` id that doesn't match the mirror id — 401 Unauthorized
- Stale `*.lastUpdated` files after a network blip — builds keep failing until `-U` or deletion
- Committing a `settings.xml` with real tokens to the repo

✓ TIPS & BEST PRACTICES

- See the merged settings: `mvn help:effective-settings`
- Keep a `.mvn/maven.config` line like `-s .mvn/settings.xml` for team-wide (secret-free) settings
- Browse and search Central at central.sonatype.com

QUICK SUMMARY

- `~/.m2/repository` — local cache
- `<mirror>` — reroute to a proxy
- `<server>` — credentials by id
- `-U` — force update checks

Build Lifecycles & Phases

THREE LIFECYCLES, MANY PHASES

Maven has three built-in **lifecycles**: `clean` wipes `target/`, `default` builds and ships, and `site` generates documentation. Each is an ordered list of **phases**. Asking for a phase runs every phase before it in the same lifecycle. Phases do nothing on their own: the project's `packaging` binds plugin goals to them, and you can bind more.

NOTES

- The main phases: `validate · compile · test · package · verify · install · deploy`
- `verify` = package + integration tests + checks
- Show what runs: `mvn help:describe -Dcmd=verify`

UNDER THE HOOD

- `packaging` decides the default bindings (jar vs pom vs war)
- Phases are just names; goals do the work, so an empty phase is a no-op
- `clean` is its own lifecycle: `mvn verify` never cleans

WHAT EACH COMMAND RUNS

```
mvn clean           # pre-clean → clean
mvn compile         # validate → initialize → generate-sources
                    # → process-sources → generate-resources
                    # → process-resources → compile
mvn test            # … → process-classes → test-compile → test
mvn package         # … → prepare-package → package
mvn verify          # … → pre-integration-test → integration-test
                    # → post-integration-test → verify
mvn install         # … → install   (copy to ~/.m2)
mvn deploy          # … → deploy    (upload to remote repo)

mvn clean verify    # two lifecycles, left to right

# see the plan for a phase
mvn help:describe -Dcmd=package
# 'package' is a phase corresponding to this plugin:
# org.apache.maven.plugins:maven-jar-plugin:3.4.1:jar
```

CODE EXPLAINED

- → `generate-sources` — where code generators (OpenAPI, jOOQ) bind
- → `process-resources` — copies and filters `src/main/resources`
- → `pre/post-integration-test` — start / stop a server or container
- → Exact plugin versions depend on your Maven version

GRADLE EQUIVALENT

- Phases → lifecycle tasks: `assemble`, `check`, `build`
- `mvn install` → `publishToMavenLocal`
- Binding a goal to a phase → `dependsOn` / `finalizedBy`
- See Gradle 01 · Gradle 07

IDIOMATIC MAVEN

- ✗ ~~`mvn clean install` in CI~~ → ✓ `mvn verify` **(deploy when releasing)**
- ✗ ~~`mvn clean` before every build~~ → ✓ **only when output is stale or renamed**
- ✗ ~~`mvn package` to run integration tests~~ → ✓ `mvn verify`

JAR PACKAGING BINDINGS

PHASE	GOAL
`process-resources`	resources:resources
`compile`	compiler:compile
`process-test-resources`	resources:testResources
`test-compile`	compiler:testCompile
`test`	surefire:test
`package`	jar:jar
`install / deploy`	install:install / deploy:deploy

OTHER LIFECYCLES & PACKAGING

ITEM	PHASES / BINDINGS
`clean`	pre-clean → clean → post-clean
`site`	pre-site → site → post-site → site-deploy
`pom packaging`	install, deploy only
`war packaging`	jar:jar replaced by war:war

⚠ COMMON MISTAKES

- Expecting `mvn deploy` to skip tests — it runs every earlier phase, tests included
- Running `mvn install` just to compile — slower and fills `~/.m2` with SNAPSHOTs
- Binding work to `integration-test` and calling `mvn integration-test` — `post-integration-test` cleanup never runs; call `verify`

✓ TIPS & BEST PRACTICES

- Use `mvn validate` as a fast "is the POM OK" check
- Get a list of every phase with `mvn help:describe -Dcmd=deploy`
- Learn the seven main phases; the in-between ones matter only when binding plugins

QUICK SUMMARY

- `clean / default / site` — three lifecycles
- Phase — runs all earlier phases
- Packaging — decides default goal bindings
- `verify` — the CI phase

Configuring Plugins

PLUGINS DO THE WORK

Plugins are configured under `<build><plugins>`. Plugin-wide settings go in `<configuration>`. To run a goal at a specific point in the build, add an `<execution>` with an `id`, a `phase` and the `goals`. `<pluginManagement>` sets versions and defaults **without** activating a plugin, which is how parents share plugin setup.

NOTES

- Always pin plugin versions; Maven warns when they're missing
- Core plugins use `groupId` `org.apache.maven.plugins` (may be omitted)
- Configuration inside an execution applies to that execution only

UNDER THE HOOD

- Each goal is a **Mojo** class; `@Parameter` fields receive the `<configuration>` XML
- Executions with the same `id` merge across parent and child
- Built-in bindings use ids like `default-compile`; CLI calls use `default-cli`

PLUGINMANAGEMENT, CONFIGURATION, EXECUTIONS

```
<build>
  <pluginManagement>                        <!-- versions + defaults -->
    <plugins>
      <plugin>
        <artifactId>maven-surefire-plugin</artifactId>
        <version>3.5.3</version>
      </plugin>
    </plugins>
  </pluginManagement>

  <plugins>                                 <!-- actually used -->
    <plugin>
      <groupId>org.codehaus.mojo</groupId>
      <artifactId>build-helper-maven-plugin</artifactId>
      <version>3.6.1</version>
      <executions>
        <execution>
          <id>add-generated</id>
          <phase>generate-sources</phase>
          <goals><goal>add-source</goal></goals>
          <configuration>
            <sources><source>target/generated/java</source></sources>
          </configuration>
        </execution>
      </executions>
    </plugin>
  </plugins>
</build>
```

CODE EXPLAINED

→ Surefire is bound by default; `pluginManagement` only pins its version

→ `<id>` — unique name; lets children override this execution

→ `<phase>` — when the goal runs; many goals have a default phase

→ Check what a goal accepts: `help:describe -Dplugin=… -Dgoal=… -Ddetail`

GRADLE EQUIVALENT

- `<plugin>` → `plugins { id("…") version "…" }`
- `pluginManagement` → `pluginManagement` / `apply false`
- An execution → a task wired with `dependsOn`
- See Gradle 06

IDIOMATIC MAVEN

✗ ~~plugins without `<version>`~~ → ✓ **versions in** `pluginManagement`

✗ ~~`maven-antrun-plugin` scripts~~ → ✓ **a purpose-built plugin**

✗ ~~same execution config in 10 modules~~ → ✓ **parent** `pluginManagement`

PLUGIN ELEMENTS

ELEMENT	PURPOSE
`<version>`	Pin it for reproducible builds
`<configuration>`	Settings for all executions
`<executions>`	Goal-to-phase bindings
`<dependencies>`	Extra classpath for the plugin itself
`<inherited>false`	Don't pass to children
`<extensions>true`	Plugin adds packaging / lifecycle

INSPECT PLUGINS

COMMAND	SHOWS
`help:describe -Dplugin=jar`	Goals of a plugin
`… -Dgoal=jar -Ddetail`	Every parameter
`help:effective-pom`	Merged plugin config
`versions:display-plugin-updates`	Newer versions

⚠ COMMON MISTAKES

- Configuring a plugin in `pluginManagement` only and wondering why it never runs
- Two executions with no `<id>` — both become `default` and merge into one
- Using `<configuration>` names from a blog for a different plugin version

✓ TIPS & BEST PRACTICES

- Add the enforcer's `requirePluginVersions` rule to catch unpinned plugins
- Override a built-in binding by reusing its id (`default-jar`)
- Disable an inherited execution with `<phase>none</phase>`

QUICK SUMMARY

- `<plugins>` — plugins that run
- `<pluginManagement>` — versions and defaults only
- `<execution>` — bind goals to phases
- `help:describe` — find parameters

Compiler & Resources

COMPILING JAVA, COPYING RESOURCES

The **compiler plugin** runs `javac` on `src/main/java` and `src/test/java`. Set the target Java version with `<release>`, which also checks you only use APIs from that version. Annotation processors such as Lombok and MapStruct go on `annotationProcessorPaths`. The **resources plugin** copies `src/main/resources` into `target/classes` and can replace `${…}` placeholders when filtering is on.

NOTES

- Since JDK 23, `javac` doesn't run processors found on the classpath by default
- `<release>` needs JDK 9+; it replaces `source` + `target`
- Toolchains compile with a different JDK than the one running Maven

UNDER THE HOOD

- The compiler plugin runs `javac` in-process through `javax.tools` (no fork by default)
- Change detection is coarse; a stale `target/` can hide errors, so `clean` when in doubt
- Filtering replaces `${}` during `process-resources`, before compiling

COMPILER AND RESOURCES SETUP

```xml
<build>
  <resources>
    <resource>
      <directory>src/main/resources</directory>
      <filtering>true</filtering>          <!-- ${project.version} → 2.4.0 -->
    </resource>
  </resources>
  <plugins>
    <plugin>
      <artifactId>maven-compiler-plugin</artifactId>
      <version>3.14.0</version>
      <configuration>
        <release>21</release>
        <compilerArgs>
          <arg>-Xlint:all</arg>
          <arg>-parameters</arg>
        </compilerArgs>
        <annotationProcessorPaths>
          <path>
            <groupId>org.projectlombok</groupId>
            <artifactId>lombok</artifactId>
            <version>1.18.38</version>
          </path>
        </annotationProcessorPaths>
      </configuration>
    </plugin>
  </plugins>
</build>
```

CODE EXPLAINED

- → `<filtering>true` — `app.version=${project.version}` gets the real value
- → `<release>21` — same as the `maven.compiler.release` property
- → `-parameters` — keeps parameter names (Spring, Jackson use them)
- → `annotationProcessorPaths` — runs Lombok without leaking it to runtime

GRADLE EQUIVALENT

- `<release>` → `options.release = 21` or a toolchain
- `annotationProcessorPaths` → `annotationProcessor(…)`
- Filtering → `tasks.processResources { expand(…) }`
- See Gradle 13 · Gradle 14

IDIOMATIC MAVEN

- ✗ ~~`<source>21</source><target>21</target>`~~ → ✓ `<release>21</release>`
- ✗ ~~filtering the whole resources folder~~ → ✓ **a separate filtered folder**
- ✗ ~~Lombok only as a `provided` dependency~~ → ✓ **also on `annotationProcessorPaths`**

COMPILER OPTIONS

OPTION	EFFECT
`<release>`	Target version + API check
`<compilerArgs>`	Raw javac flags
`<parameters>true`	Same as `-parameters`
`<showWarnings>true`	Print warnings
`<failOnWarning>true`	Warnings break the build
`<fork>true`	Run javac in a separate process

FILTERING NOTES

ITEM	DETAIL
`Delimiters`	`${x}` and `@x@`
`Binary files`	Exclude, or set `nonFilteredFileExtensions`
`toolchains.xml`	`~/.m2/toolchains.xml` lists installed JDKs
`<jdkToolchain>`	Compiler plugin picks a JDK by version

⚠ COMMON MISTAKES

- Filtering fonts or images — the bytes get corrupted
- Lombok "cannot find symbol" errors on JDK 23+ — add it to `annotationProcessorPaths`
- Spring Boot apps filtering `application.properties` — Boot's own `${}` placeholders get replaced too; Boot's parent uses `@x@`

✓ TIPS & BEST PRACTICES

- Put `-Xlint:all` and `failOnWarning` in the parent to keep code clean
- Use `@project.version@` in Boot projects to avoid placeholder clashes
- Run `mvn clean compile` after switching branches with big refactors

QUICK SUMMARY

- `<release>` — target Java version
- `annotationProcessorPaths` — Lombok, MapStruct
- `<filtering>` — replace `${}` in resources
- toolchains — compile with another JDK

Packaging & Running JARs

FROM CLASSES TO A RUNNABLE JAR

`mvn package` creates a plain JAR with your classes only, so dependencies are not inside it. To run it with `java -jar`, set a `Main-Class` in the manifest **and** either ship the dependencies alongside it or build an **uber JAR** with the **shade plugin**. For quick runs during development, `exec:java` runs a main class with the full classpath.

NOTES

- Spring Boot apps use `spring-boot-maven-plugin` (`repackage`)
- `war` packaging runs `war:war` instead of `jar:jar`
- Libraries publish `-sources` and `-javadoc` JARs too

MAIN-CLASS + SHADED UBER JAR

```
<plugin>
  <artifactId>maven-shade-plugin</artifactId>
  <version>3.6.0</version>
  <executions>
    <execution>
      <phase>package</phase>
      <goals><goal>shade</goal></goals>
      <configuration>
        <transformers>
          <transformer implementation=
            "org.apache.maven.plugins.shade.resource.ManifestResourceTransformer">
            <mainClass>com.acme.App</mainClass>
          </transformer>
          <transformer implementation=
            "org.apache.maven.plugins.shade.resource.ServicesResourceTransformer"/>
        </transformers>
      </configuration>
    </execution>
  </executions>
</plugin>

<!-- mvn package && java -jar target/invoice-service-2.4.0.jar -->
<!-- dev run: mvn compile exec:java -Dexec.mainClass=com.acme.App -->
```

CODE EXPLAINED

- → `shade` — unpacks every dependency into one JAR at `package`
- → `ManifestResourceTransformer` — writes `Main-Class`
- → `ServicesResourceTransformer` — merges `META-INF/services` files
- → `exec:java` — runs in Maven's JVM with the project classpath

GRADLE EQUIVALENT

- `Main-Class` → `application { mainClass = "…" }`
- Shade plugin → Shadow plugin (`shadowJar`)
- `exec:java` → `./gradlew run`
- See Gradle 15

IDIOMATIC MAVEN

- ✗ ~~shading a library you publish~~ → ✓ **shade only applications**
- ✗ ~~assembly `jar-with-dependencies`~~ → ✓ **shade (handles service files)**
- ✗ ~~`java -cp` with a hand-built path~~ → ✓ **shade or `dependency:copy-dependencies`**

PACKAGING PLUGINS

PLUGIN	BUILDS
`maven-jar-plugin`	Plain JAR + manifest
`maven-shade-plugin`	Uber JAR, can relocate packages
`maven-assembly-plugin`	zip / tar.gz distributions
`maven-war-plugin`	WAR for a servlet container
`maven-source-plugin`	`-sources.jar` (`jar-no-fork`)
`maven-javadoc-plugin`	`-javadoc.jar`

PLAIN JAR MANIFEST

JAR-PLUGIN SETTING	EFFECT
`<mainClass>`	Main-Class entry
`<addClasspath>true`	Class-Path of dependency JARs
`<classpathPrefix>lib/`	Where those JARs sit
`dependency:copy-dependencies`	Copies them to `target/dependency`

⚠ COMMON MISTAKES

- `java -jar` on the plain JAR — "no main manifest attribute" or `NoClassDefFoundError`
- Shading without `ServicesResourceTransformer` — JDBC drivers or SLF4J providers vanish
- Signed dependency JARs inside a shaded JAR — `SecurityException` ; filter out `*.SF` , `*.RSA`

✓ TIPS & BEST PRACTICES

- Use `<relocations>` to rename clashing packages in a shaded JAR
- Set `<createDependencyReducedPom>false` if the reduced POM confuses tools
- For containers, a thin JAR plus a dependency layer builds faster than an uber JAR

QUICK SUMMARY

- `jar:jar` — classes only
- `shade` — runnable uber JAR
- `exec:java` — run during development
- Source / javadoc JARs — for published libraries

Sheet 17

Testing with Surefire & Failsafe

maven-surefire-plugin · JUnit 5 · -Dtest=Class#method · -DskipTests vs -Dmaven.test.skip · failsafe *IT classes · integration-test / verify · test reports

UNIT TESTS AND INTEGRATION TESTS

Surefire runs unit tests in the `test` phase and fails the build right away on a failure. **Failsafe** runs integration tests (classes named `*IT`) in `integration-test` but only fails the build in `verify`, so `post-integration-test` cleanup (stopping a server or container) still runs. Both detect JUnit 5 automatically when `junit-jupiter` is on the test classpath.

NOTES

- Surefire is bound by default; Failsafe must be added
- Reports: `target/surefire-reports`, `target/failsafe-reports`
- JUnit and Mockito APIs: Java sheets 102–104

UNDER THE HOOD

- Surefire forks a separate test JVM (`forkCount=1`, `reuseForks=true`)
- `<argLine>` sets that JVM's flags; JaCoCo injects its agent through `@{argLine}`
- Failsafe records results in `integration-test` and fails in `verify`

FAILSAFE SETUP AND TEST COMMANDS

```
<plugin>
  <artifactId>maven-surefire-plugin</artifactId>
  <version>3.5.3</version>
  <configuration>
    <argLine>@{argLine} -Xmx1g</argLine>   <!-- keeps JaCoCo's agent -->
  </configuration>
</plugin>
<plugin>
  <artifactId>maven-failsafe-plugin</artifactId>
  <version>3.5.3</version>
  <executions>
    <execution>
      <goals>
        <goal>integration-test</goal>
        <goal>verify</goal>
      </goals>
    </execution>
  </executions>
</plugin>

mvn test                              # unit tests only
mvn verify                            # unit + integration tests
mvn test -Dtest=InvoiceServiceTest    # one class
mvn test -Dtest='InvoiceServiceTest#creates*'   # methods by pattern
mvn verify -Dit.test=PaymentIT        # one integration test
mvn verify -DskipITs                  # skip Failsafe only
```

CODE EXPLAINED

→ `@{argLine}` — late replacement, so JaCoCo's agent isn't overwritten

→ Both Failsafe goals are needed: one runs tests, one fails the build

→ `-Dtest` — Surefire filter; `-Dit.test` — Failsafe filter

→ Quote patterns with `#` or `*` in the shell

GRADLE EQUIVALENT

- `mvn test` → `./gradlew test` ; `-Dtest=X` → `--tests X`
- Failsafe → a `jvm-test-suite` named `integrationTest`
- `-DskipTests` → `-x test`
- See Gradle 16

IDIOMATIC MAVEN

✗ ~~`-Dmaven.test.skip=true` by default~~ → ✓ `-DskipTests` **(tests still compile)**

✗ ~~slow container tests named `*Test`~~ → ✓ **name them `*IT` for Failsafe**

✗ ~~`<argLine>-Xmx1g</argLine>` with JaCoCo~~ → ✓ `@{argLine} -Xmx1g`

DEFAULT NAME PATTERNS

PLUGIN	RUNS CLASSES NAMED
Surefire	**/Test*, **/*Test, **/*Tests, **/*TestCase
Failsafe	**/IT*, **/*IT, **/*ITCase

SKIPPING & FILTERING

PROPERTY	EFFECT
`-DskipTests`	Compile tests, run none
`-Dmaven.test.skip=true`	Don't compile or run tests
`-DskipITs`	Skip Failsafe only
`-Dsurefire.failIfNoSpecifiedTests=false`	OK if –Dtest matches nothing in a module
`-Dgroups=fast`	JUnit 5 @Tag filter
`-Dsurefire.rerunFailingTestsCount=2`	Retry flaky tests

⚠ COMMON MISTAKES

- Only declaring Failsafe's `integration-test` goal — failures never break the build
- `-Dtest=X` in a multi-module build fails in modules without X — add `-Dsurefire.failIfNoSpecifiedTests=false`
- Test classes named `*Spec` or `Check*` — the default patterns skip them silently; add `<includes>`

✓ TIPS & BEST PRACTICES

- Speed up: `<forkCount>1C</forkCount>` runs one fork per CPU core
- Readable reports: `mvn surefire-report:report`
- Watch out for "Tests run: 0" in CI; it usually means a naming or engine problem

QUICK SUMMARY

- Surefire — unit tests in `test`
- Failsafe — `*IT` tests, fails in `verify`
- `-Dtest=Class#method` — run a subset
- `-DskipTests` — skip running, keep compiling

Code Quality & Enforcer

GUARDRAILS IN THE BUILD

Quality plugins turn team rules into build failures. The **enforcer plugin** checks the environment and the dependency graph: Maven and Java versions, pinned plugins, banned or conflicting dependencies. **JaCoCo** measures test coverage and can enforce a minimum. Formatters and linters like **Spotless** and **Checkstyle** keep style consistent. Bind the checks to `verify` so CI runs them.

NOTES

- Put these in the parent POM so every module gets them
- Spotless: `spotless:apply` fixes, `spotless:check` fails
- JaCoCo report: `target/site/jacoco/index.html`

ENFORCER + JACOCO

```
<plugin>
  <artifactId>maven-enforcer-plugin</artifactId>
  <version>3.5.0</version>
  <executions><execution>
    <id>enforce</id>
    <goals><goal>enforce</goal></goals>
    <configuration><rules>
      <requireMavenVersion><version>[3.9,)</version></requireMavenVersion>
      <requireJavaVersion><version>[21,)</version></requireJavaVersion>
      <requirePluginVersions/>
      <dependencyConvergence/>
      <bannedDependencies><excludes>
        <exclude>commons-logging:commons-logging</exclude>
      </excludes></bannedDependencies>
    </rules></configuration>
  </execution></executions>
</plugin>
<plugin>
  <groupId>org.jacoco</groupId>
  <artifactId>jacoco-maven-plugin</artifactId>
  <version>0.8.13</version>
  <executions>
    <execution><id>agent</id><goals><goal>prepare-agent</goal></goals></execution>
    <execution><id>report</id><phase>verify</phase><goals><goal>report</goal></goals></execution>
  </executions>
</plugin>
```

CODE EXPLAINED

- → `enforce` — binds to `validate` by default, so it fails fast
- → `[3.9,)` — version range: 3.9 or newer
- → `dependencyConvergence` — fails when two paths bring different versions
- → `prepare-agent` — sets `argLine` so tests run with coverage

GRADLE EQUIVALENT

- Enforcer → no built-in; use `failOnVersionConflict()` and plugins
- JaCoCo → the `jacoco` plugin + `jacocoTestReport`
- Spotless / Checkstyle → same tools as Gradle plugins
- See Gradle 06

IDIOMATIC MAVEN

- ✗ ~~"use Java 21" in the README~~ → ✓ `requireJavaVersion`
- ✗ ~~style debates in code review~~ → ✓ `spotless:check` in CI
- ✗ ~~coverage numbers no one reads~~ → ✓ a JaCoCo `check` rule with a floor

USEFUL ENFORCER RULES

RULE	FAILS WHEN
`requireMavenVersion`	Maven too old
`requireJavaVersion`	JDK too old
`requirePluginVersions`	A plugin has no version
`dependencyConvergence`	Conflicting transitive versions
`requireUpperBoundDeps`	A lower version beat a higher one
`bannedDependencies`	A listed artifact appears

QUALITY PLUGINS

PLUGIN	GOAL
`jacoco-maven-plugin`	report · check
`spotless-maven-plugin`	apply · check
`maven-checkstyle-plugin`	check
`spotbugs-maven-plugin`	check
`dependency-check-maven`	check (CVE scan)
`versions-maven-plugin`	display-*-updates

⚠ COMMON MISTAKES

- Adding `dependencyConvergence` to a big legacy project at once — hundreds of failures; start with `requireUpperBoundDeps`
- Overriding Surefire's `argLine` without `@{argLine}` — JaCoCo shows 0% coverage
- Running Spotless `apply` in CI — it changes files instead of failing

✓ TIPS & BEST PRACTICES

- Run `mvn enforcer:enforce` alone to test rules quickly
- Use `spotless:apply` in a pre-commit hook and `spotless:check` in CI
- Start the JaCoCo floor at your current coverage, then raise it

QUICK SUMMARY

- Enforcer — environment and dependency rules
- JaCoCo — coverage report and floor
- Spotless / Checkstyle — formatting and style
- Bind checks to `verify`

Multi-Module Projects

ONE BUILD, MANY MODULES

A **multi-module** project has a root POM with `packaging pom` that lists child folders in `<modules>`. Building the root builds every module in one **reactor** run, in dependency order. The root is usually both the **aggregator** (lists modules) and the **parent** (shares config), but the two roles are separate. Modules depend on each other with normal `<dependency>` entries.

NOTES

- Use `${project.version}` for sibling dependencies
- CI-friendly `${revision}` sets one version for all modules
- Maven 3 needs `flatten-maven-plugin` to publish `${revision}` POMs

UNDER THE HOOD

- The reactor sorts modules by their dependency graph, not by `<modules>` order
- `-am` adds the upstream modules you depend on; `-amd` the downstream ones
- In-build module output wins over older copies in `~/.m2`

ROOT POM + MODULE

```
<!-- root pom.xml -->
<groupId>com.acme</groupId>
<artifactId>billing</artifactId>
<version>${revision}</version>
<packaging>pom</packaging>
<properties><revision>2.4.0-SNAPSHOT</revision></properties>
<modules>
  <module>core</module>
  <module>api</module>
  <module>app</module>
</modules>

<!-- app/pom.xml -->
<parent>
  <groupId>com.acme</groupId><artifactId>billing</artifactId>
  <version>${revision}</version>
</parent>
<artifactId>app</artifactId>
<dependencies>
  <dependency><groupId>com.acme</groupId><artifactId>core</artifactId>
    <version>${project.version}</version></dependency>
</dependencies>

mvn verify                    # whole reactor
mvn verify -pl app -am        # app + what it needs
mvn verify -pl core -amd      # core + everything using it
mvn verify -rf :api           # resume after a failure
```

CODE EXPLAINED

- → `packaging pom` — the root builds no JAR
- → `${revision}` — change the version in one place
- → `-pl app` — select by folder or `:artifactId`
- → `-rf :api` — skip modules that already passed

GRADLE EQUIVALENT

- `<modules>` → `include("core", "api", "app")`
- Sibling dependency → `implementation(project(":core"))`
- `-pl app -am` → `./gradlew :app:build`
- See Gradle 17

IDIOMATIC MAVEN

- ✗ ~~`mvn install` so siblings can see each other~~ → ✓ `mvn verify` **from the root**
- ✗ ~~hard-coded sibling versions~~ → ✓ `${project.version}`
- ✗ ~~`versions:set` on every release~~ → ✓ **CI-friendly** `-Drevision=2.4.0`

REACTOR FLAGS

FLAG	EFFECT
`-pl a,b`	Only these modules
`-pl !c`	All except c
`-am`	Also make upstream deps
`-amd`	Also make downstream dependents
`-rf :x`	Resume from module x
`-T 1C`	Build independent modules in parallel

AGGREGATOR VS PARENT

ROLE	MEANS
Aggregator	Lists `<modules>`, builds them together
Parent	Children inherit its config
Both	The usual root POM
Maven 4	`<subprojects>` replaces `<modules>`

⚠ COMMON MISTAKES

- Running `mvn test` inside one module folder — siblings resolve from stale `~/.m2` copies
- A dependency cycle between modules — "The projects in the reactor contain a cyclic reference"
- Deploying `${revision}` POMs without flattening in Maven 3 — consumers can't resolve them

✓ TIPS & BEST PRACTICES

- Run one module's tests from the root: `mvn test -pl app -am`
- Use `flatten-maven-plugin` with `flattenMode=resolveCiFriendliesOnly`
- Keep a `-bom` module listing every module for consumers

QUICK SUMMARY

- `<modules>` — build children together
- Reactor — orders modules by dependencies
- `-pl / -am / -amd` — build a slice
- `${revision}` — one version for all

Deploying & Releasing

PUBLISHING ARTIFACTS

`mvn deploy` uploads the built artifacts to the repository named in `<distributionManagement>`: SNAPSHOT versions go to the snapshot repository and releases to the release repository. Publishing open source to **Maven Central** goes through the **Central Portal** (OSSRH was shut down in 2025) and requires sources, javadoc, GPG signatures and complete POM metadata.

NOTES

- Credentials come from `settings.xml` `<server>` by id
- Central needs `name`, `description`, `url`, `licenses`, `developers`, `scm`
- A release version can never be overwritten on Central

UNDER THE HOOD

- `deploy` uploads the artifact, POM, checksums and `maven-metadata.xml`
- SNAPSHOTs get unique timestamped versions on the remote: `2.4.0-20261026.141503-7`
- `install` only copies to `~/.m2`; nothing leaves your machine

COMPANY REPO + CENTRAL PORTAL

```
<distributionManagement>                    <!-- company Nexus -->
  <repository>
    <id>corp-releases</id>
    <url>https://nexus.acme.com/repository/maven-releases/</url>
  </repository>
  <snapshotRepository>
    <id>corp-snapshots</id>
    <url>https://nexus.acme.com/repository/maven-snapshots/</url>
  </snapshotRepository>
</distributionManagement>

<plugin>                                    <!-- Maven Central -->
  <groupId>org.sonatype.central</groupId>
  <artifactId>central-publishing-maven-plugin</artifactId>
  <version>0.8.0</version>
  <extensions>true</extensions>
  <configuration>
    <publishingServerId>central</publishingServerId>
    <autoPublish>false</autoPublish>
  </configuration>
</plugin>

mvn versions:set -DnewVersion=2.4.0 -DgenerateBackupPoms=false
mvn -Prelease deploy           # sources, javadoc, gpg in the profile
mvn versions:set -DnewVersion=2.5.0-SNAPSHOT
```

CODE EXPLAINED

- → `<id>` — must match a `<server>` in `settings.xml`
- → `<extensions>true` — the plugin replaces the default deploy step
- → `autoPublish=false` — review the upload in the Portal before releasing
- → `versions:set` — updates the version in every module

GRADLE EQUIVALENT

- `distributionManagement` → `publishing { repositories { } }`
- `mvn deploy` → `./gradlew publish`
- GPG plugin → the `signing` plugin
- See Gradle 23

IDIOMATIC MAVEN

- ✗ ~~deploying from a laptop~~ → ✓ **deploy from CI on a tag**
- ✗ ~~manual version edits in each POM~~ → ✓ `versions:set` **or** `${revision}`
- ✗ ~~signing every dev build~~ → ✓ **GPG + javadoc only in a** `release` **profile**

CENTRAL REQUIREMENTS

ITEM	PLUGIN
`-sources.jar`	maven-source-plugin `jar-no-fork`
`-javadoc.jar`	maven-javadoc-plugin `jar`
`.asc signatures`	maven-gpg-plugin `sign`
`POM metadata`	name, url, licenses, developers, scm
`Verified namespace`	groupId you own, set up in the Portal

RELEASE PLUGIN GOALS

GOAL	DOES
`release:prepare`	Drop SNAPSHOT, tag, bump to next
`release:perform`	Check out the tag and deploy
`release:rollback`	Undo a failed prepare
`release:clean`	Remove release backup files

⚠ COMMON MISTAKES

- Deploying a SNAPSHOT to a release-only repository — 400 Bad Request
- Still pointing at `oss.sonatype.org` (OSSRH) — it's retired; use the Central Portal plugin
- Missing GPG passphrase in CI — set `MAVEN_GPG_PASSPHRASE` as a secret

✓ TIPS & BEST PRACTICES

- Test a release locally with `mvn -Prelease install` before deploying
- Use `-DaltDeploymentRepository=id::url` to deploy elsewhere once
- Many teams skip `release:prepare` and let CI set the version from the Git tag

QUICK SUMMARY

- `mvn deploy` — upload to a remote repo
- `distributionManagement` — where to upload
- Central Portal — the way into Maven Central
- `versions:set` — change the version

What's Changing in Maven 4

MAVEN 4 AT A GLANCE

Maven 4 is a major release that keeps Maven 3 POMs building while cleaning up long-standing pain points. It splits the **build POM** (what you write) from the **consumer POM** (what gets published), so build-only details stop leaking to users. A new POM model, `4.1.0`, allows shorter POMs. As of October 2026 it is at release candidate 4.0.0-rc-7, with GA expected soon.

A MAVEN 4.1.0 POM

```
<project xmlns="http://maven.apache.org/POM/4.1.0">
  <modelVersion>4.1.0</modelVersion>

  <parent>
    <!-- groupId / version inferred from ../pom.xml -->
    <artifactId>billing</artifactId>
  </parent>
  <artifactId>app</artifactId>

  <dependencies>
    <dependency>
      <groupId>com.acme</groupId>
      <artifactId>core</artifactId>   <!-- sibling: version inferred -->
    </dependency>
  </dependencies>
</project>

<!-- root: <subprojects><subproject>app</subproject></subprojects> -->

mvnup check                # report upgrade issues
mvnup apply                # rewrite POMs where safe
mvn -r verify              # resume after a failed build
```

NOTES

- Running Maven 4 needs **Java 17+**; it can still compile older targets
- Existing `4.0.0` POMs keep working; most builds just run
- `mvnup check` lists problems; `mvnup apply` fixes them

CODE EXPLAINED

- → `4.1.0` — opt in to the new model; publishing stays compatible
- → Parent and sibling versions can be left out inside one build
- → `<subprojects>` — new name for `<modules>`
- → `-r` / `--resume` — continues where the last build failed

GRADLE EQUIVALENT

- Consumer POM → Gradle Module Metadata + generated POM
- Inferred versions → `project(":core")` never needed versions
- No direct `mvnup` counterpart
- See Gradle 23

IDIOMATIC MAVEN

- ✗ ~~`flatten-maven-plugin`~~ for ~~`${revision}`~~ → **✓ Maven 4 handles CI-friendly versions itself**
- ✗ ~~repeating parent~~ ~~`groupId`~~ / ~~`version`~~ → **✓ inferred in a 4.1.0 POM**
- ✗ ~~upgrading POMs by hand~~ → **✓ `mvnup apply`, then review the diff**

KEY CHANGES

AREA	MAVEN 4
Runtime	Java 17+ to run Maven
POM model	4.1.0 (4.0.0 still read)
Published POM	Consumer POM, build details removed
Modules	`<subprojects>`; `<modules>` deprecated
Phases	`before:` / `after:` hooks on every phase
Packaging	New `bom` packaging

UPGRADE CHECKLIST

STEP	WHY
Run on JDK 17+	Required to start Maven 4
Update plugins	Old plugins may use removed APIs
mvnup check	Finds POM problems
Try with the wrapper	`-Dmaven=4.0.0` on a branch
Check extensions	Ask authors about Maven 4 support

⚠ COMMON MISTAKES

- Switching the POM to `4.1.0` while teammates still run Maven 3 — Maven 3 can't read it
- Assuming every plugin works — very old plugins that use removed Maven 2 APIs fail
- Upgrading CI and laptops separately — pin the version with the wrapper instead

✓ TIPS & BEST PRACTICES

- Trial Maven 4 on a branch: `mvn wrapper:wrapper -Dmaven=4.0.0-rc-7`
- Run with `-e` and report plugin failures to the plugin's project
- Read "What's new in Maven 4" on maven.apache.org before upgrading

QUICK SUMMARY

- Java 17+ — required to run Maven 4
- Consumer POM — clean published POMs
- `4.1.0` — shorter POMs, inferred versions
- `mvnup` — upgrade helper

CI & Troubleshooting

BUILDS THAT RUN ANYWHERE

In CI, run Maven in **batch mode** through the wrapper, cache `~/.m2/repository` between runs, and use `verify` as the main command. When a build fails, add –e for stack traces or –X for full debug output. Most "works on my machine" failures come from a stale local repository, a different JDK, or a different Maven version.

NOTES

- The cache key should include every `pom.xml`
- For faster local builds, try `mvnd` (Maven Daemon)
- The build cache extension skips modules whose inputs haven't changed

GITHUB ACTIONS WORKFLOW

```yaml
name: build
on: [push, pull_request]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v5
      - uses: actions/setup-java@v5
        with:
          distribution: temurin
          java-version: 21
          cache: maven              # caches ~/.m2/repository
      - run: ./mvnw -B -ntp verify
      - uses: actions/upload-artifact@v4
        if: failure()
        with:
          name: test-reports
          path: '**/target/surefire-reports/'

# local troubleshooting
./mvnw -e verify                    # stack traces
./mvnw -X verify > build.log        # full debug log
./mvnw -U verify                    # re-check remote repos
./mvnw dependency:purge-local-repository   # re-download deps
```

CODE EXPLAINED

→ `cache: maven` — keyed on the hash of every `pom.xml`

→ `-B -ntp` — no colors or prompts, no download progress lines

→ Upload Surefire reports on failure so you can read them

→ `purge-local-repository` — fixes corrupt or partial downloads

GRADLE EQUIVALENT

- `cache: maven` → `gradle/actions/setup-gradle`
- `-X` → `--debug` ; `-e` → `--stacktrace`
- `-U` → `--refresh-dependencies`
- See Gradle 25

IDIOMATIC MAVEN

✗ ~~`mvn clean install` in CI~~ → ✓ `./mvnw -B -ntp verify`

✗ ~~deleting all of `~/.m2`~~ → ✓ `purge-local-repository` **or** `-U`

✗ ~~re-running failed CI to "see if it passes"~~ → ✓ **read** `surefire-reports` **first**

COMMON ERRORS

MESSAGE	USUAL FIX
`Could not resolve dependencies`	Check mirror / credentials, then –U
`was cached in the local repository`	Delete `*.lastUpdated` or use –U
`release version 21 not supported`	Maven runs on an older JDK; fix `JAVA_HOME`
`No plugin found for prefix`	Typo, or add the plugin's `groupId`
`PKIX path building failed`	Corporate proxy cert; import it into the JDK
`NoSuchMethodError at runtime`	Version conflict: `dependency:tree -Dverbose`

FASTER BUILDS

OPTION	GAIN
`-T 1C`	Parallel modules
`mvnd`	Warm daemon JVM, parallel by default
`maven-build-cache-extension`	Skip unchanged modules
`-o`	No remote checks
`-pl x -am`	Build only what you need

⚠ COMMON MISTAKES

- Caching `~/.m2` including your own SNAPSHOTs — stale artifacts leak between branches
- CI using a different JDK than developers — pin `java-version` and use `requireJavaVersion`
- Ignoring "[WARNING]" lines about missing plugin versions — builds change when Maven upgrades

✓ TIPS & BEST PRACTICES

- Reproduce CI locally with a clean repo: `-Dmaven.repo.local=/tmp/m2`
- Turn on `-Dstyle.color=always` if your CI shows ANSI colors
- Add `-fae` in CI to see every failing module, not just the first

QUICK SUMMARY

- `./mvnw -B -ntp verify` — the CI command
- `cache: maven` — reuse downloads
- `-e` / `-X` — more error detail
- `-U` / purge — fix a stale local repo

Sheet 23

Quick Reference: CLI & Lifecycle

common commands · CLI flags · lifecycle phases · plugin:goal cheat list · reactor flags · wrapper commands

COMMON COMMANDS

MVN …	DOES
clean	Delete target/
compile	Compile main code
test	Unit tests (Surefire)
package	Build the JAR / WAR
verify	Package + ITs + checks
install	Copy to ~/.m2
deploy	Upload to remote repo
dependency:tree	Dependency tree
dependency:analyze	Unused / undeclared deps
help:effective-pom	Merged POM
help:active-profiles	Active profiles
versions:display-dependency-updates	Newer versions
exec:java	Run –Dexec.mainClass

DEFAULT LIFECYCLE (MAIN PHASES)

PHASE	TYPICAL GOAL (JAR)
validate	enforcer:enforce
generate-sources	code generators
process-resources	resources:resources
compile	compiler:compile
test-compile	compiler:testCompile
test	surefire:test
package	jar:jar
pre-integration-test	start server / container
integration-test	failsafe:integration-test
post-integration-test	stop server / container
verify	failsafe:verify, jacoco:report
install	install:install
deploy	deploy:deploy

COMMAND-LINE FLAGS

FLAG	EFFECT
-D name=value	Set a property
-P a,!b	Profiles on / off
-o / -U	Offline / force updates
-q / -X / -e	Quiet / debug / stack traces
-B / -ntp	Batch mode / no progress
-T 1C	Parallel, 1 thread per core
-f path/pom.xml	Use another POM
-s settings.xml	Use another settings file
-N	Root POM only
-fae / -ff	Fail at end / fast
-DskipTests	Don't run tests
-Dmaven.test.skip=true	Don't compile tests
-v	Maven + JDK versions

REACTOR (MULTI-MODULE)

FLAG	EFFECT
-pl a,b	Only modules a, b
-pl !c	Everything except c
-am	Plus upstream dependencies
-amd	Plus downstream dependents
-rf :x	Resume from module x
-r (Maven 4)	Resume after last failure

WRAPPER & LOCAL REPO

COMMAND	DOES
wrapper:wrapper -Dmaven=x	Add / upgrade wrapper
./mvnw · mvnw.cmd	Run the pinned Maven
dependency:go-offline	Pre-download for –o
dependency:purge-local-repository	Re-download
-Dmaven.repo.local=dir	Use a clean repo
help:effective-settings	Merged settings.xml

PLUGIN PREFIXES

PREFIX	USED FOR
compiler · surefire · failsafe	build & test
dependency · help · versions	diagnostics
jar · shade · assembly · war	packaging
enforcer · jacoco · spotless	quality
release · gpg · deploy	publishing
archetype · wrapper · exec	setup / running

Sheet 24

Quick Reference: POM Snippets

minimal pom.xml · properties block · dependency scopes · BOM import · plugin with executions · settings.xml mirror & server

MINIMAL POM.XML

```
<project xmlns="http://maven.apache.org/POM/4.0.0">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.example</groupId>
  <artifactId>my-app</artifactId>
  <version>1.0.0-SNAPSHOT</version>
  <properties>
    <maven.compiler.release>21</maven.compiler.release>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>
</project>
```

PARENT + MODULES

```
<!-- root -->
<packaging>pom</packaging>
<modules>
  <module>core</module>
  <module>app</module>
</modules>

<!-- child -->
<parent>
  <groupId>com.example</groupId>
  <artifactId>my-app</artifactId>
  <version>1.0.0-SNAPSHOT</version>
</parent>
<artifactId>app</artifactId>
```

DEPENDENCIES BY SCOPE

```
<dependency>
  <groupId>g</groupId><artifactId>a</artifactId>
  <version>1.0</version>          <!-- compile -->
</dependency>
<dependency>…<scope>provided</scope></dependency>
<dependency>…<scope>runtime</scope></dependency>
<dependency>…<scope>test</scope></dependency>
<dependency>…<optional>true</optional></dependency>
<dependency>…
  <exclusions><exclusion>
    <groupId>x</groupId><artifactId>y</artifactId>
  </exclusion></exclusions>
</dependency>
```

BOM IMPORT

```
<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>org.junit</groupId>
      <artifactId>junit-bom</artifactId>
      <version>5.13.4</version>
      <type>pom</type>
      <scope>import</scope>
    </dependency>
  </dependencies>
</dependencyManagement>
```

PLUGIN WITH AN EXECUTION

```
<build>
  <pluginManagement><plugins>
    <plugin>
      <artifactId>maven-surefire-plugin</artifactId>
      <version>3.5.3</version>
    </plugin>
  </plugins></pluginManagement>
  <plugins>
    <plugin>
      <artifactId>maven-failsafe-plugin</artifactId>
      <version>3.5.3</version>
      <executions><execution>
        <goals>
          <goal>integration-test</goal>
          <goal>verify</goal>
        </goals>
      </execution></executions>
    </plugin>
  </plugins>
</build>
```

SETTINGS.XML MIRROR & SERVER

```
<settings>
  <mirrors>
    <mirror>
      <id>corp</id>
      <mirrorOf>*</mirrorOf>
      <url>https://nexus.acme.com/repository/maven-public/</url>
    </mirror>
  </mirrors>
  <servers>
    <server>
      <id>corp</id>
      <username>${env.NEXUS_USER}</username>
      <password>${env.NEXUS_TOKEN}</password>
    </server>
  </servers>
</settings>
```